Lecture Note 8: Virtual-Memory Management

CSS225 Operating System

School of ICT, SIIT, Thammasat University

Outlines

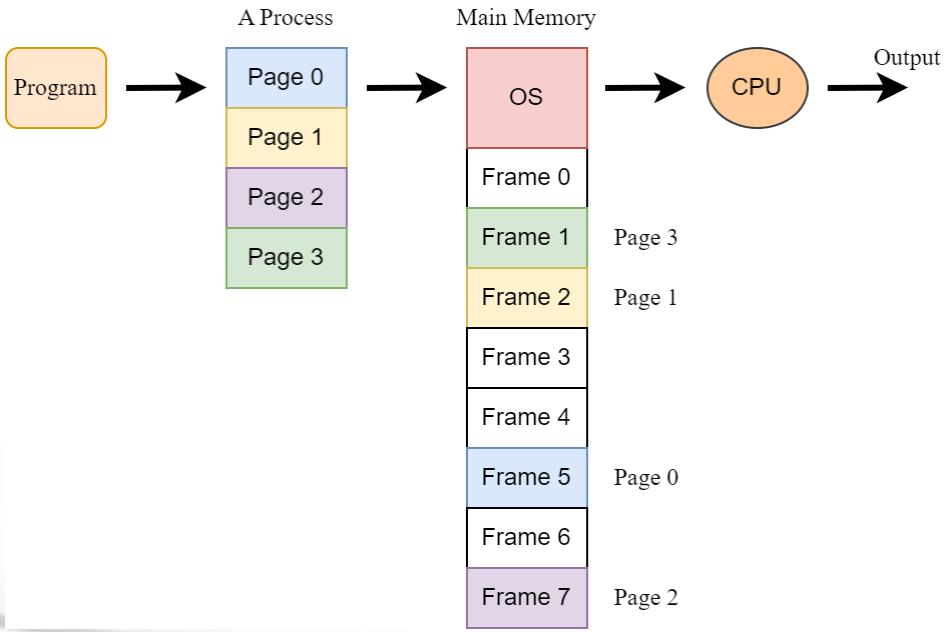
1. Virtual Memory
2. Demand Paging, Page Fault, and Page Replacement
3. Page-Replacement Algorithms and Trashing
4. Frame Allocation
5. Other Related Topics in Memory Management

1. Virtual Memory

Scenario: Paging as the Memory Management Technique

To run a process, traditionally, the process must be stored in the main memory.

1) The CPU will execute only processes (or parts of them) stored in the main memory.
2) In this lecture note, we consider a computer system using the paging method in the memory management to store processes in the main memory.
   - In the conventional paging method (Lecture 7), all pages of a process will be stored in the main memory.



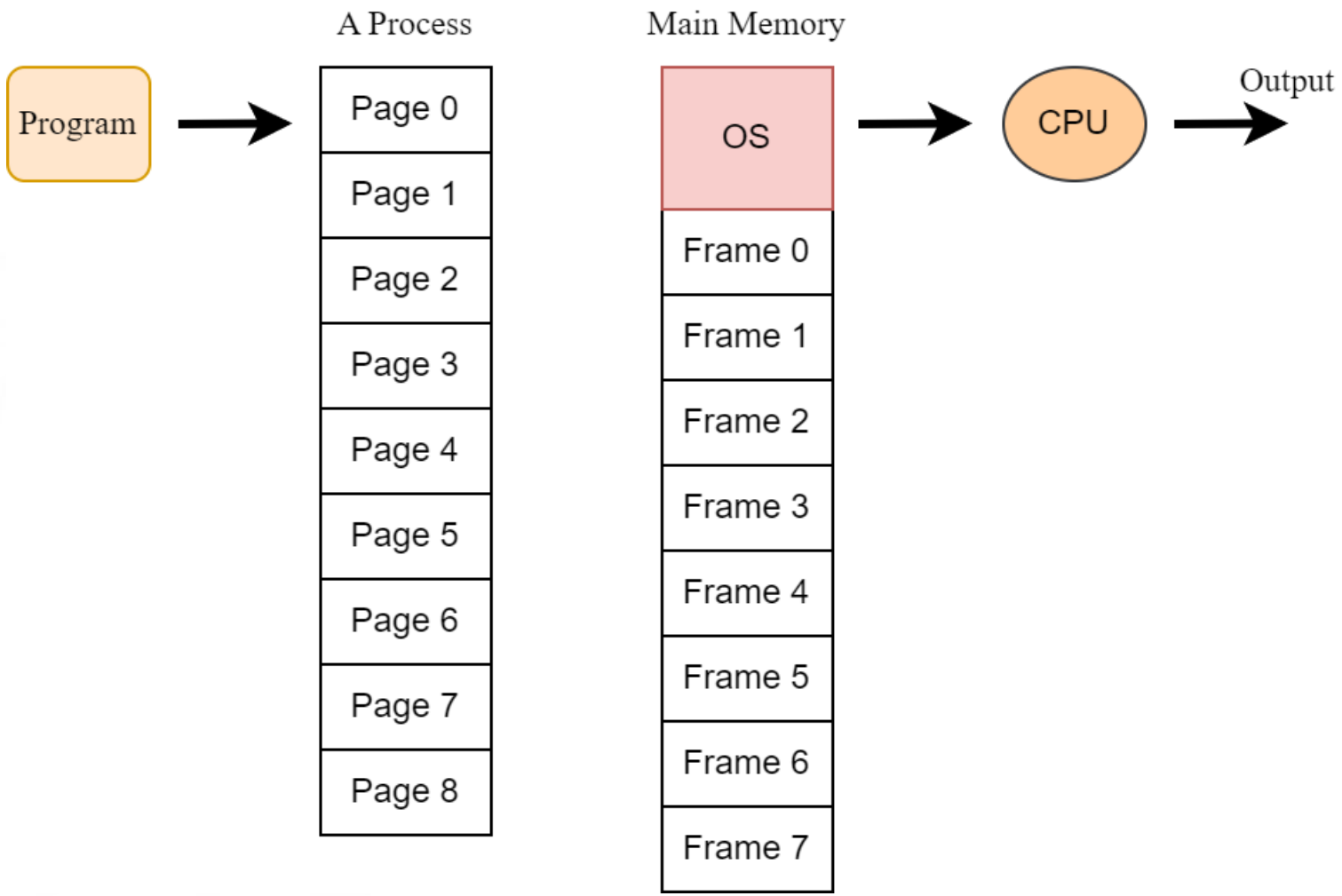
Problem Statement

By using the conventional paging method (in Lecture Note 7), where all pages of each process must be stored in the main memory, we have the following two problems.

Problem 1

A process might be larger than the available space in the main memory. As a result, we cannot run this process.

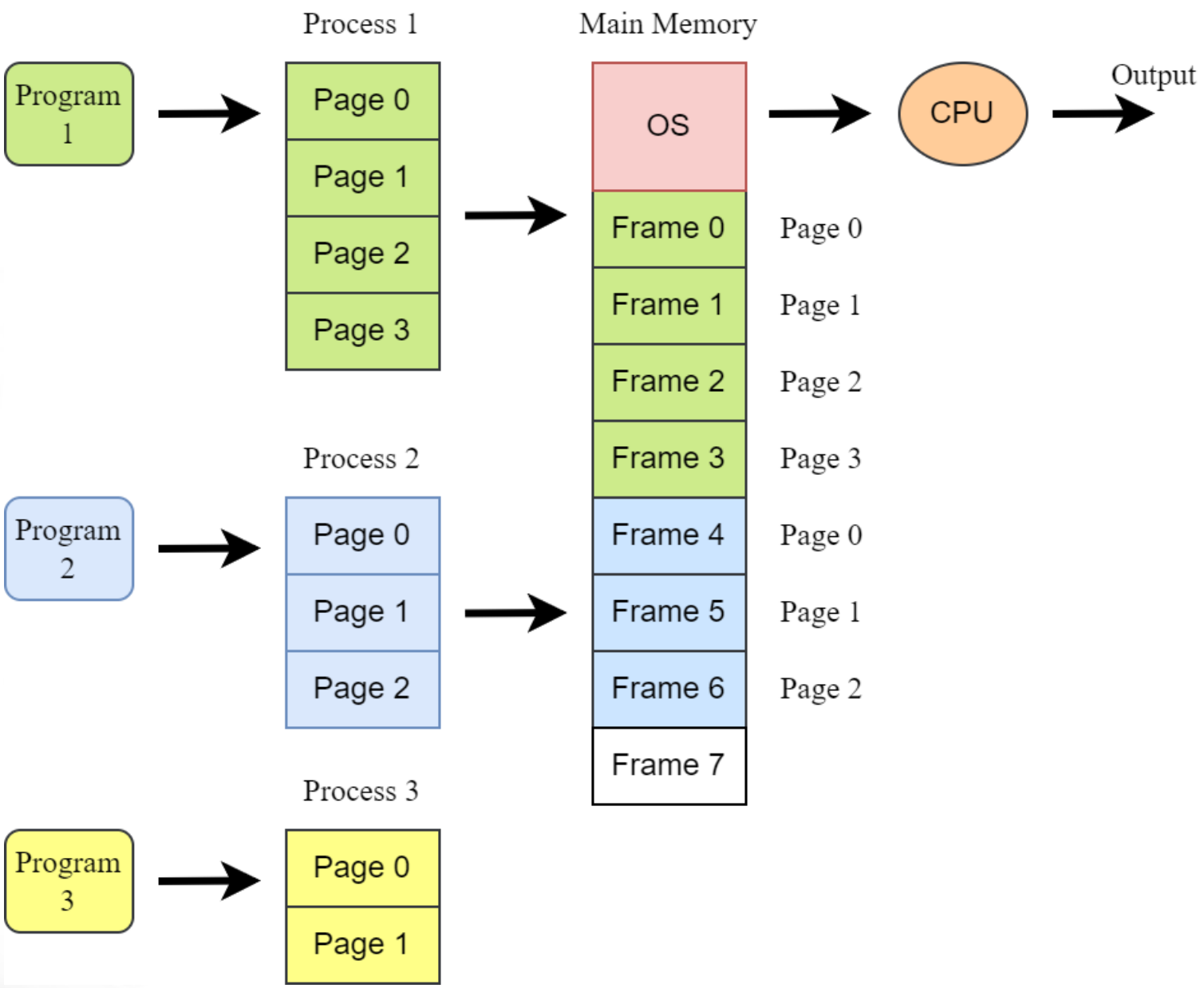
- This process consists of 9 pages (i.e., needs 9 free frames in the main memory).
- Only 8 free frames are available.
- How could we run this process?



Problem 2

For a small size of the main memory, we will be able to run only few processes (i.e., low degree of multiprogramming).

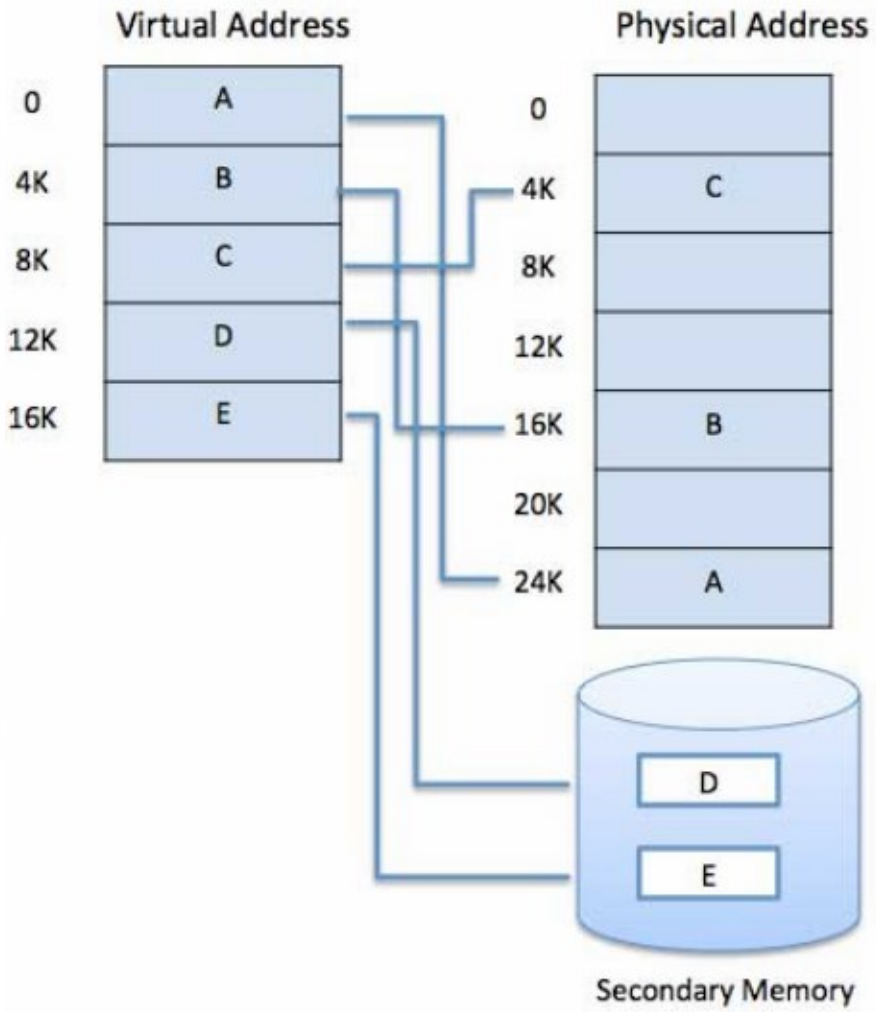
- Here, we can run only 2 processes.
- How could we run more processes (i.e., higher degree of multiprogramming)?



Solution: Virtual Memory

Virtual memory is a modern technique that allows the execution of processes that are not completely in memory; parts (i.e., pages) of a process may be stored in the hard disk. An example is shown below.

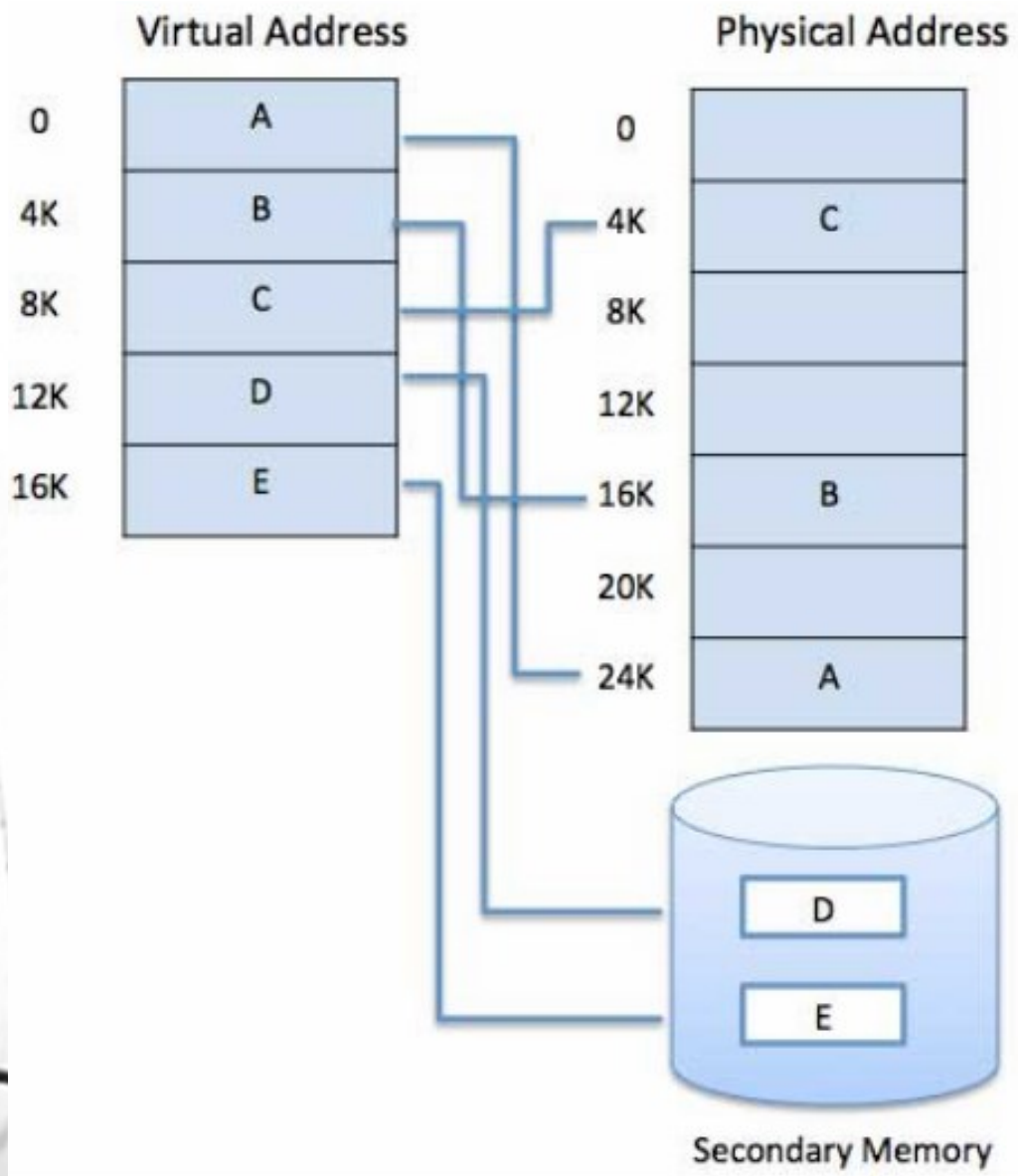
- For example, this process consists of 5 pages. By using the virtual memory technique, we have:
  - 3 pages are stored in the main memory.
  - 2 pages are stored in the hard disk.
- Note that, the virtual addresses are equivalent to logical addresses. We use the term virtual address in the topic Virtual Memory.



Virtual Memory: Advantages vs Disadvantages

1) Advantage:
   - By applying virtual memory, we can increase the number of processes (known as the degree of multiprogramming) stored in the memory.
2) Disadvantage:
   a) Additional steps to handle the case that parts of a process not in the memory must be loaded into the memory when needed:
      - demand paging
      - page fault
      - page replacement
   b) Overhead time.

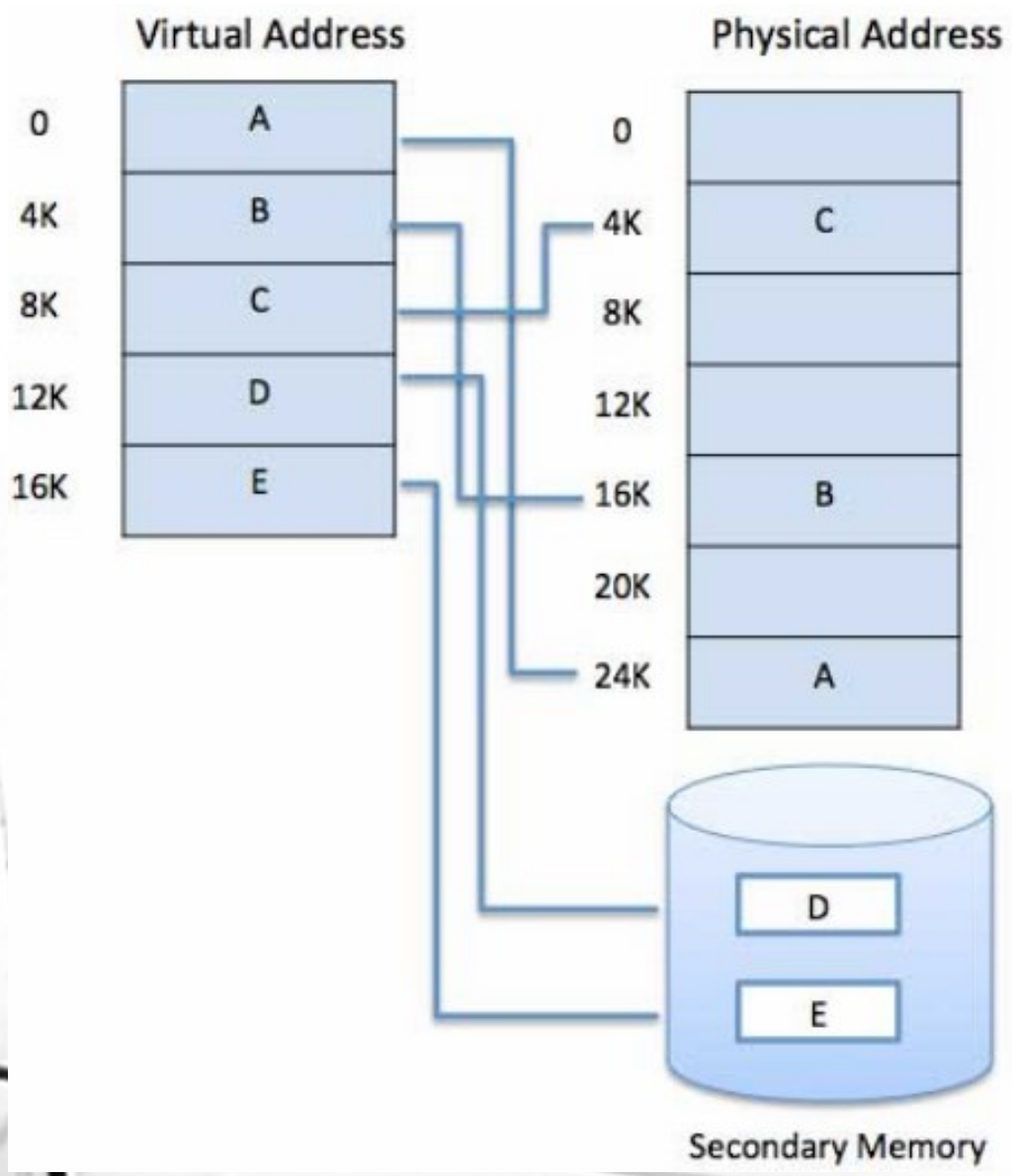
Example 1: Page Table



Draw a page table consisting of three columns (page number, frame number, valid/invalid bit) of this process.

Page	Frame	Valid/Invalid
0	6	v
1	4	v
2	1	v
3	-	i
4	-	i

Example 2: Page Table



1) Assume that the CPU needs to run Page 4 (i.e. E). We will move Page 4 from the hard disk and store it in a free frame in the main memory.
2) Assume that we store Page E in Frame 5.
3) Draw a page table consisting of three columns (page number, frame number, valid/invalid bit) of this process.

Page	Frame	Valid/Invalid
0	6	v
1	4	v
2	1	v
3	-	i
4	5	v

2. Demand Paging, Page Fault, and Page Replacement

Additional mechanisms when using the virtual memory technique:

- demand paging
- page fault
- page replacement

Demand Paging

Demand paging is a technique used in virtual memory where pages in a disk will be loaded/stored into the memory only when they are needed, not in advance (i.e., when the CPU needs to execute these pages).

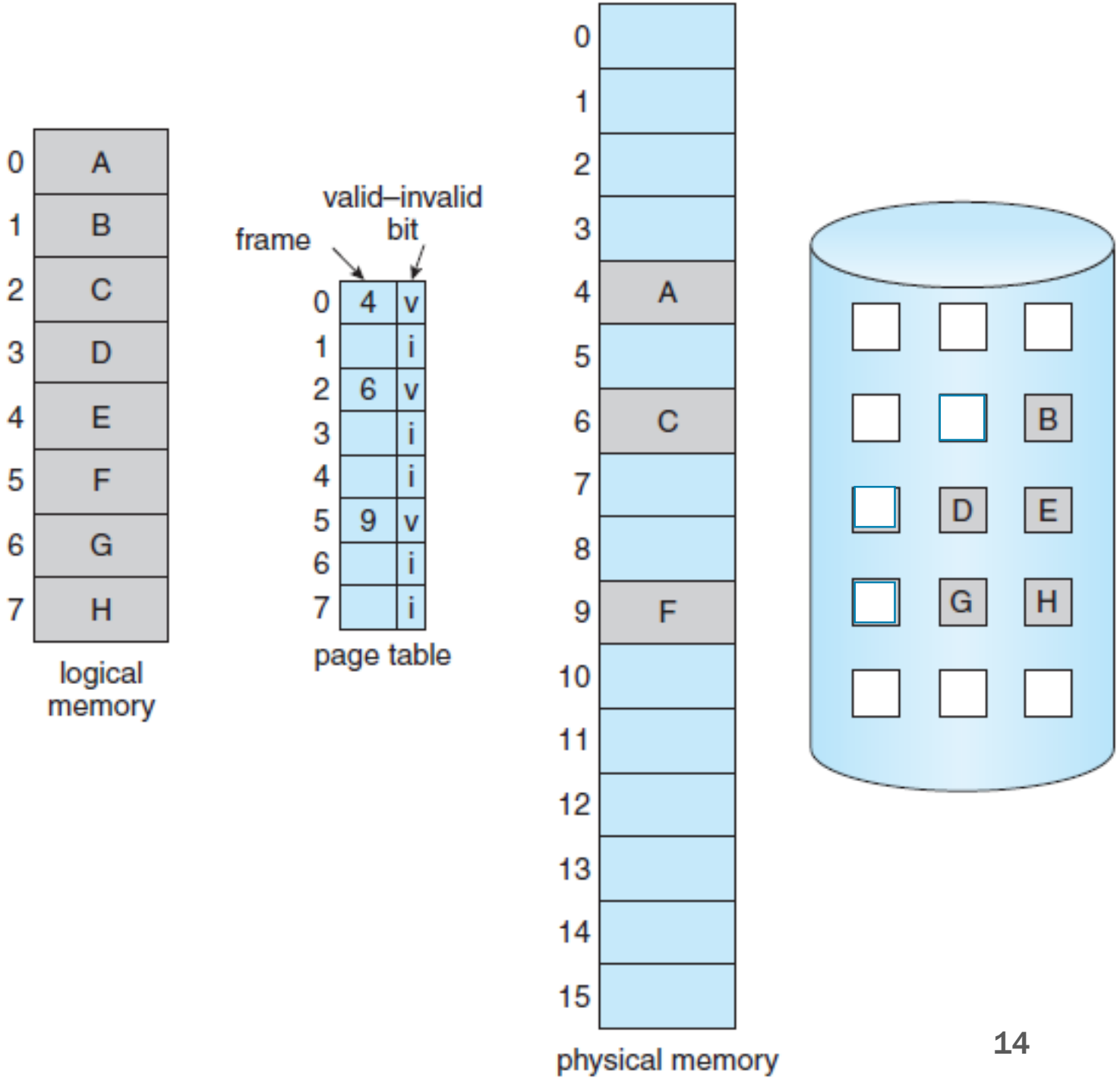
1) A unit called pager is concerned swapping a page in and out between the main memory and the hard disk.
2) The area of the hard disk that is used to store these "valid pages but in a hard disk" is called swap space.
3) A pager can know that which pages are in the memory and in the disk by checking the page table.
4) In the page table, the valid-invalid bit identifies where a page is.
   - If this bit of a page is set to valid, the page is in the memory.
   - If this bit of a page is set to invalid, the page is either "valid but in a disk" or "invalid (i.e., unused)".

Example

Virtual memory of a process consisting of 8 pages.

- Pages 0, 2, 5 are stored in the main memory.
- Pages 1, 3, 4, 6, 7 are stored in the hard disk.

By using the demand paging, if the CPU needs to execute Page 3 (D), we will move Page 3 (D) from the hard disk and store it in a free frame in the main memory.



Page Fault

During a CPU is running a process, the CPU might need to access a page in a hard disk (an invalid page).

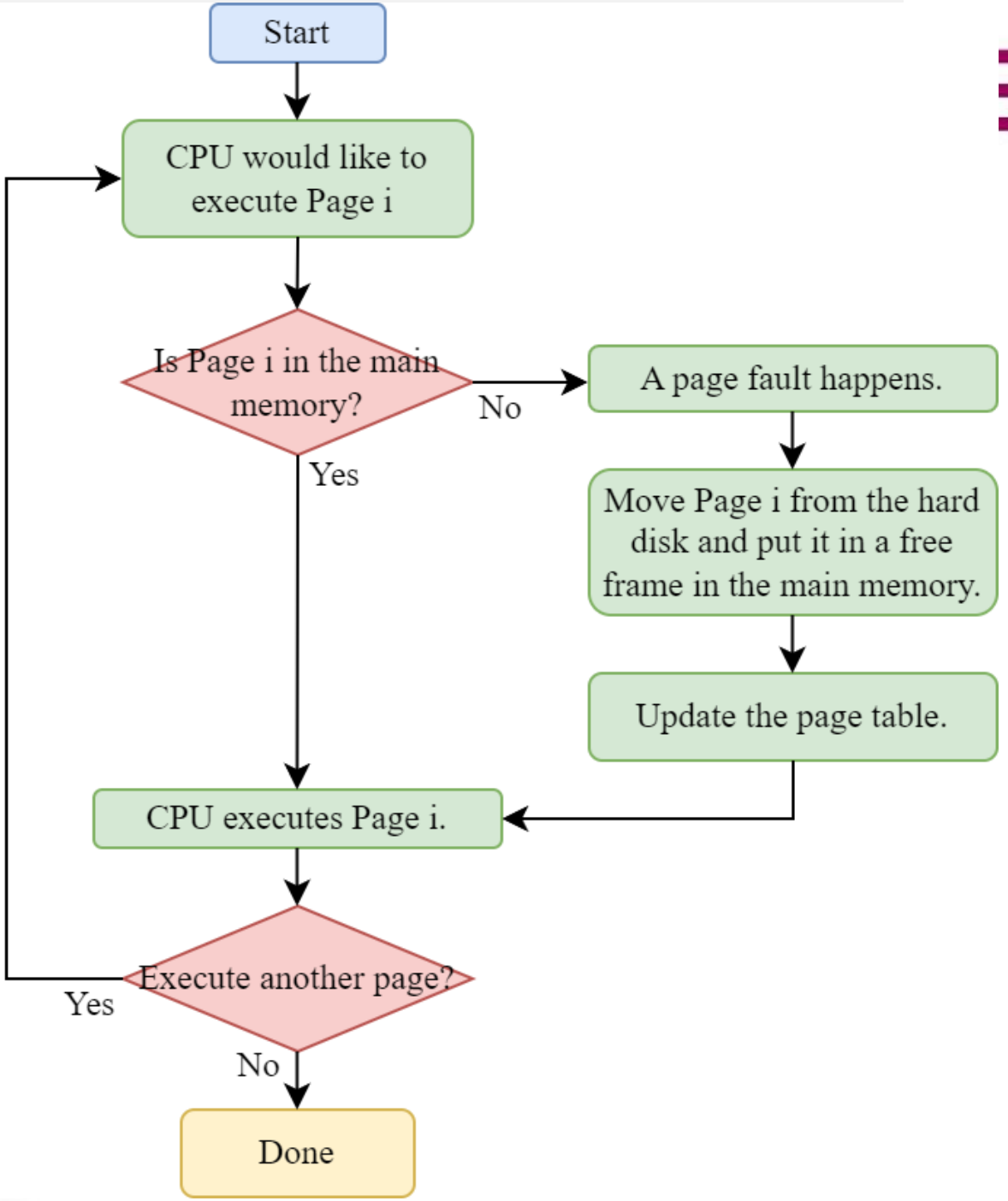
1) Access to a page marked invalid causes a page fault.
2) Then, there is an operation to transfer this page from the disk to the memory such that the CPU can execute.

The CPU may encounter page faults many time until the process terminates/finishes. The steps are explained by a flowchart in the next page.

A page fault is the situation that the CPU would like to execute a page that is in the hard disk. After having the page fault, the OS will move this page from the hard disk and store it in the main memory.

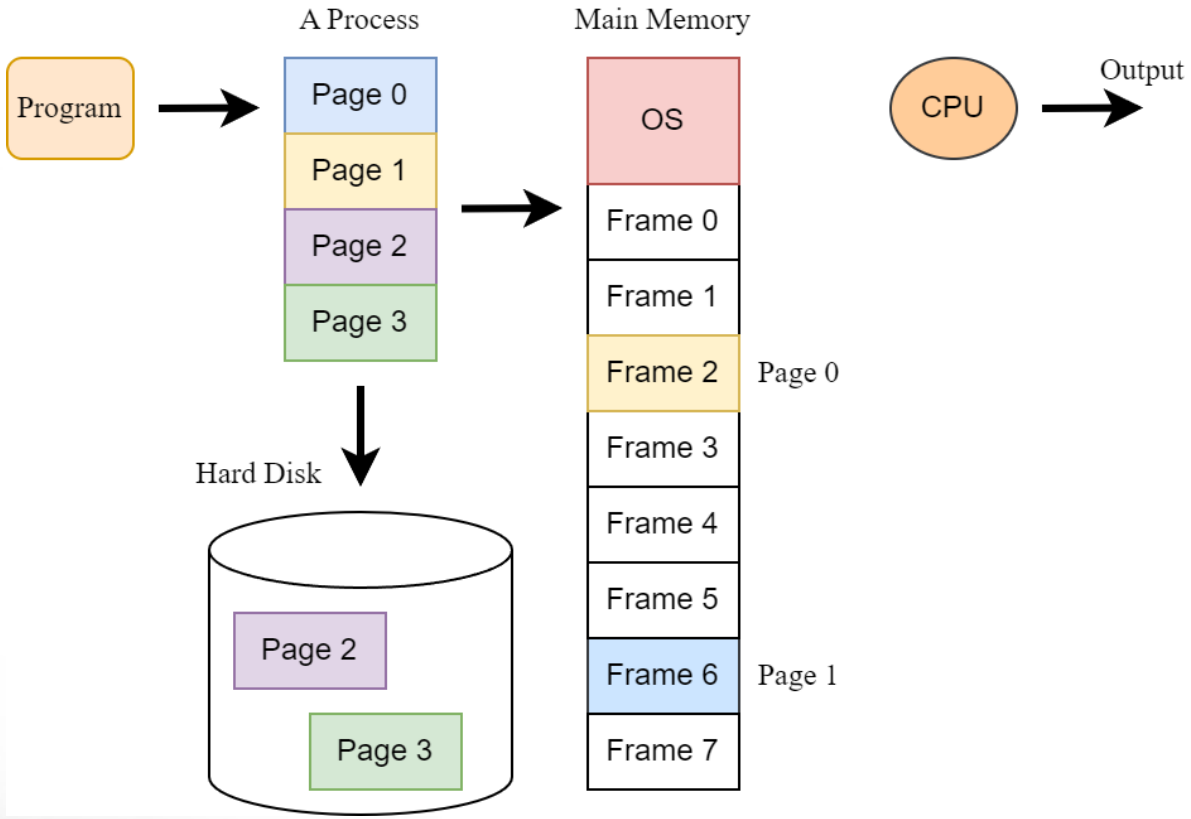
Demand-Paging Operation: Flow Chart

The steps to execute a process.



Example:

- Process P1 is divided into 4 pages and stored in the main memory and the hard disk as show below by using the virtual-memory technique.
- The CPU executes the following pages of P1 in order: Pages 0, 1, 2, 3. The following operations happen.



Page Replacement

Consider a scenario that a page fault happens.

- The CPU needs to execute a page in the hard disk. Let us call it as Page i.

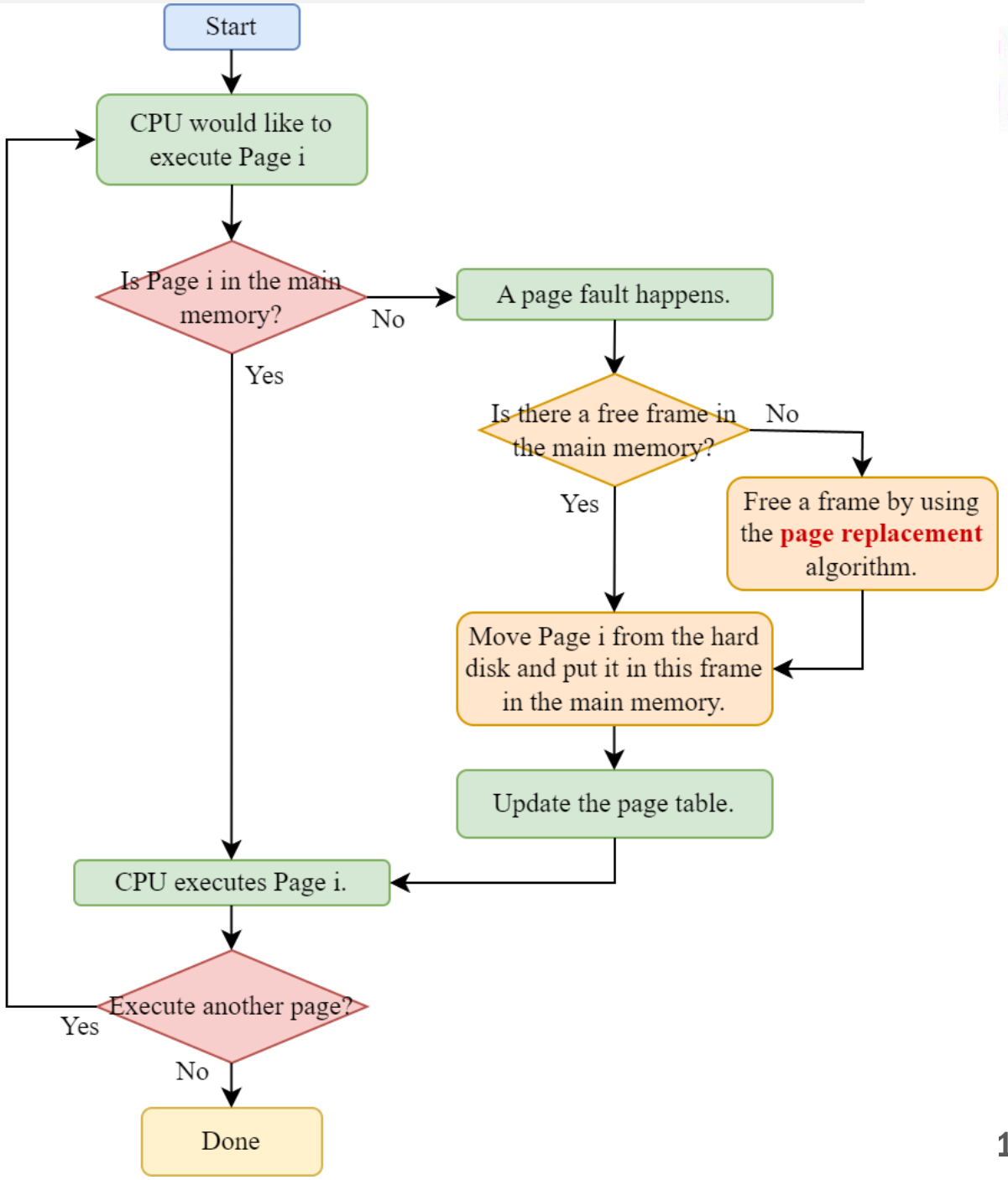
The operating system must move Page i from the hard disk and store it in a frame in the main memory. We have the following steps.

1. Find the location of Page i on the hard disk.
2. Find a free frame to store Page i:
   a) If there is a free frame, store Page i in this frame. Update the page table.
   b) If there is no free frame, use a page-replacement algorithm:
      - Find an occupied frame (victim) that is not currently being used and free it.
      - Free a frame by
        i) writing its contents to swap space (i.e., move this page to the hard disk);
        ii) changing the page table (and all other tables) to indicate that the page is no longer in memory.
      - Store Page i into the newly freed frame and update the page and frame tables.
3. The CPU executes Page i.

Demand-Paging Operation: Flow Chart

The steps to execute a process.

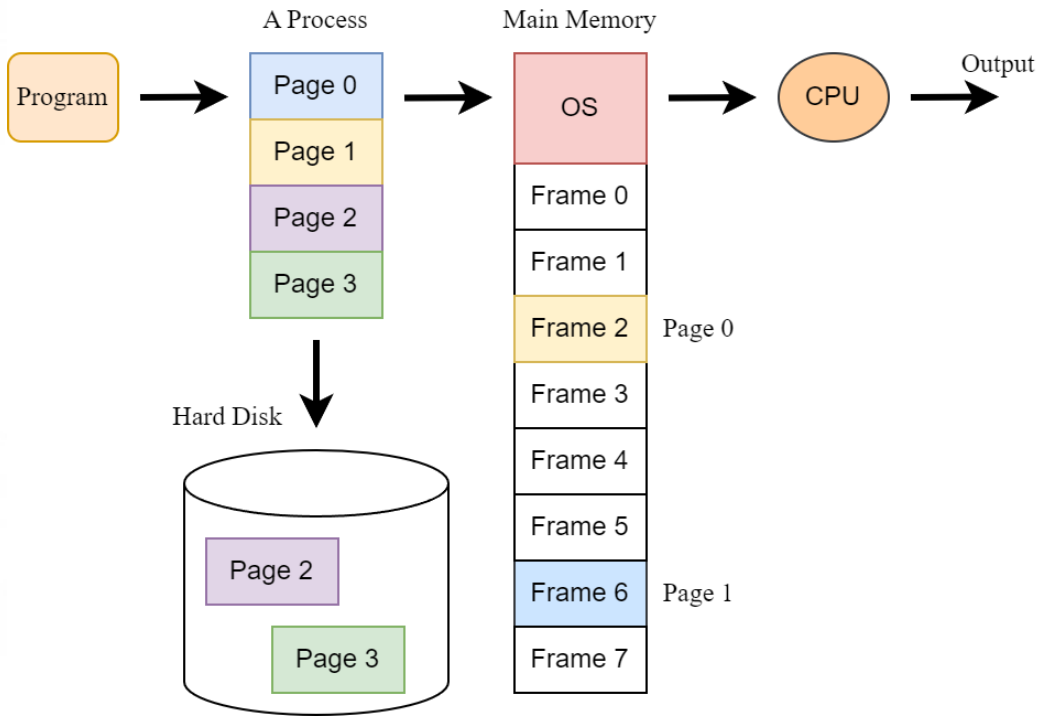
- This diagram is similar to that shown on Page 16.
- Here, we add the orange-color blocks to explain the steps during swapping pages between the main memory and the hard disk.



Example: Case that there are free frames available. No Page Replacement

- Process P1 is divided into 4 pages and stored in the main memory and the hard disk as show below by using the virtual-memory technique.
- The CPU would like to execute Page 2 of P1. Page fault happens. We follow the steps shown in the diagram on Page 19:

  1) Select a free frame.
  2) Move Page 2 from the hard disk to this selected free frame.
  3) Update the page table.
  4) CPU executes Page 2.

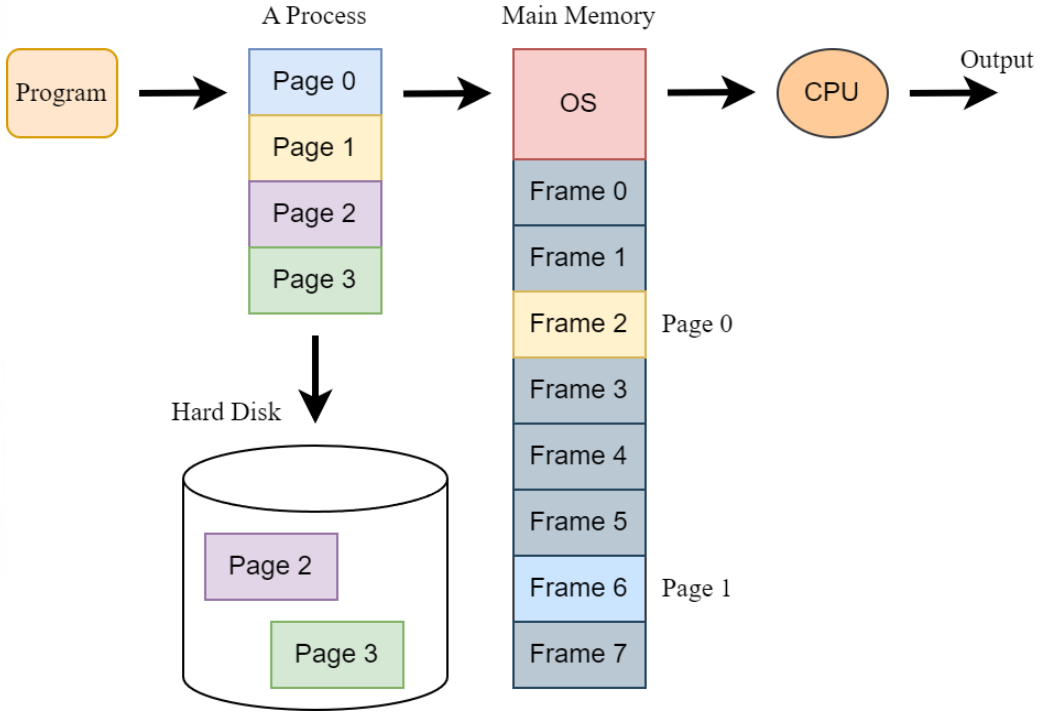


Example: Case that there is “no” free frame. Need Page Replacement

Assume all frames are occupied.

- Process P1 is divided into 4 pages and stored in the main memory and the hard disk as show below by using the virtual-memory technique.
- The CPU would like to execute Page 2 of P1. Page fault happens but there is no free frame. We need to run page replacement. We follow the steps shown in the diagram on Page 19:

1) Select an occupied frame (victim).
2) Move the page/content in this frame to the hard disk.
3) Move Page 2 from the hard disk to this selected frame.
4) Update the page table.
5) CPU executes Page 2.



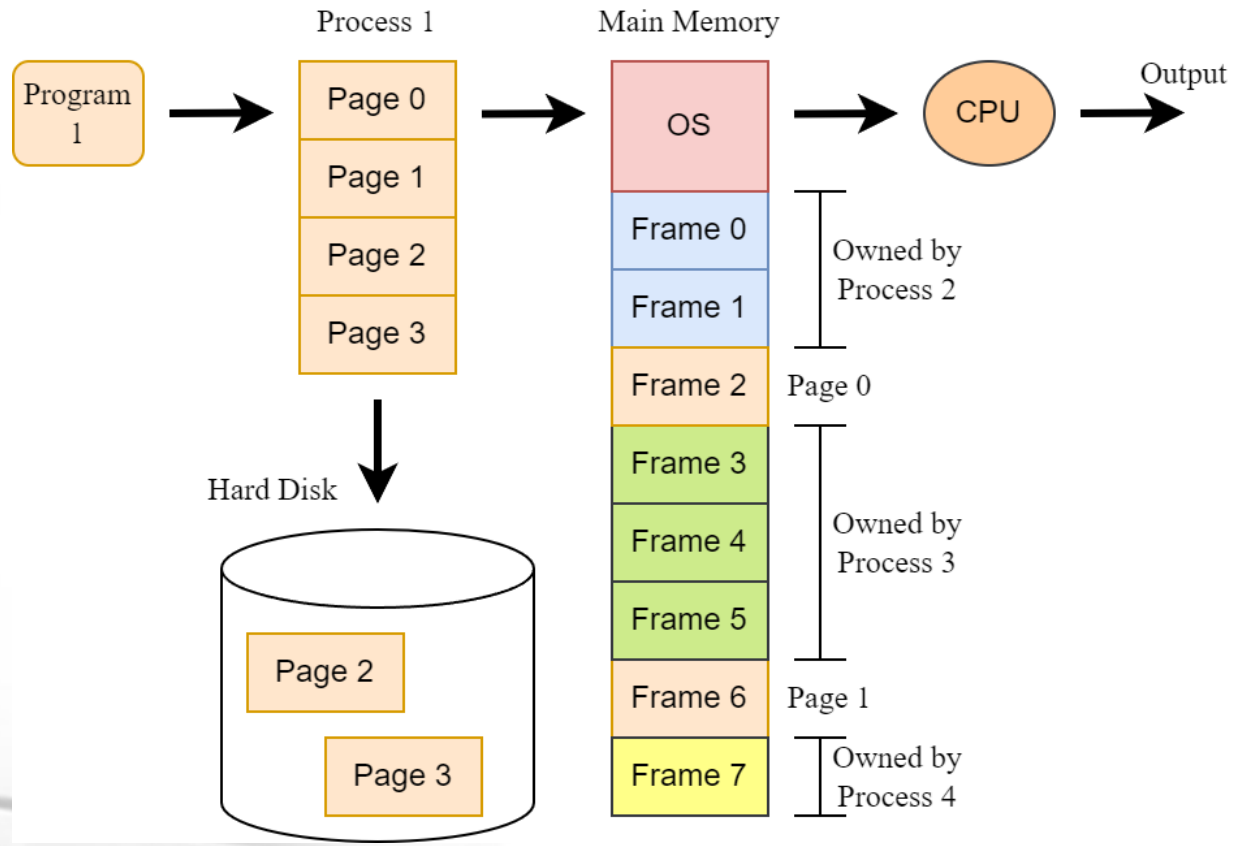
Page-Replacement Strategy

- Page-replacement algorithms can be classified into 2 strategies.
  - Global replacement allows the OS to select a replacement frame from the set of all frames, even if that frame is currently allocated to some other process.
  - Local replacement requires that each process select from only its own set of allocated frames.
    - Assigning the number of frames to a process here must be more careful.
- Generally, the global replacement generally results in greater system throughput and is therefore the more commonly used method.

Example: Global replacement vs. local replacement

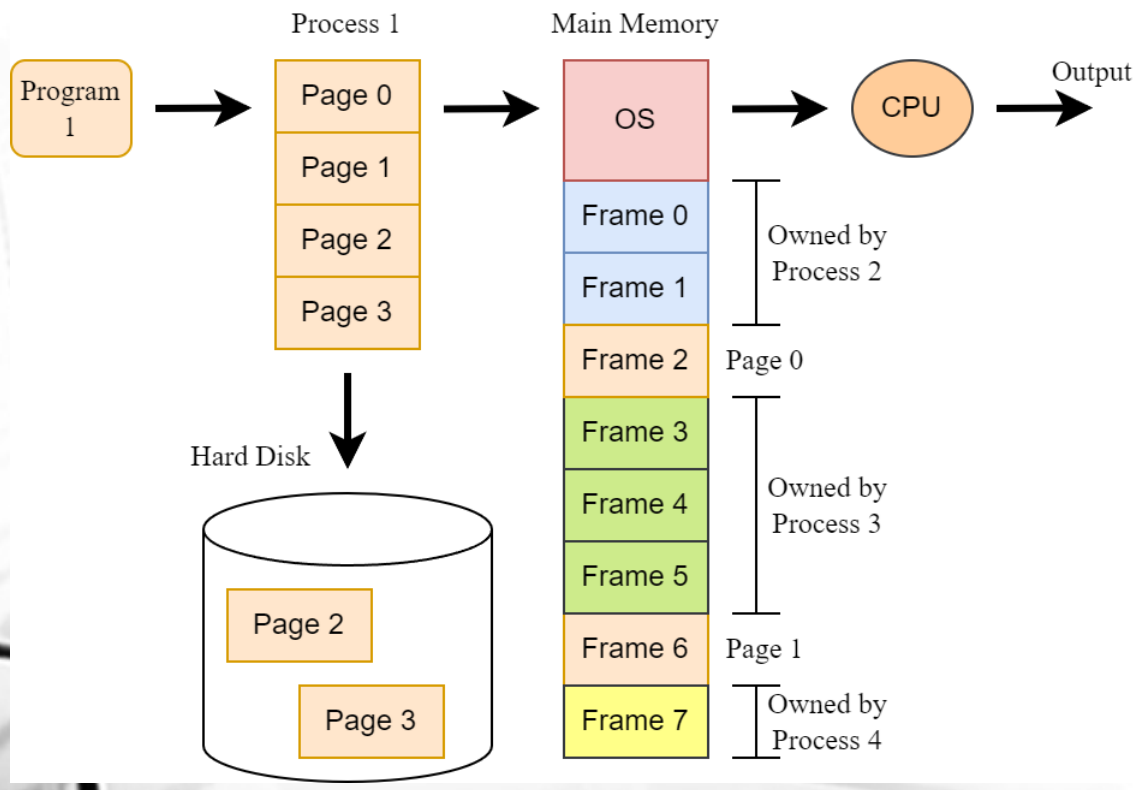
Consider the following diagram where there are four processes stored in the main memory and hard disk.

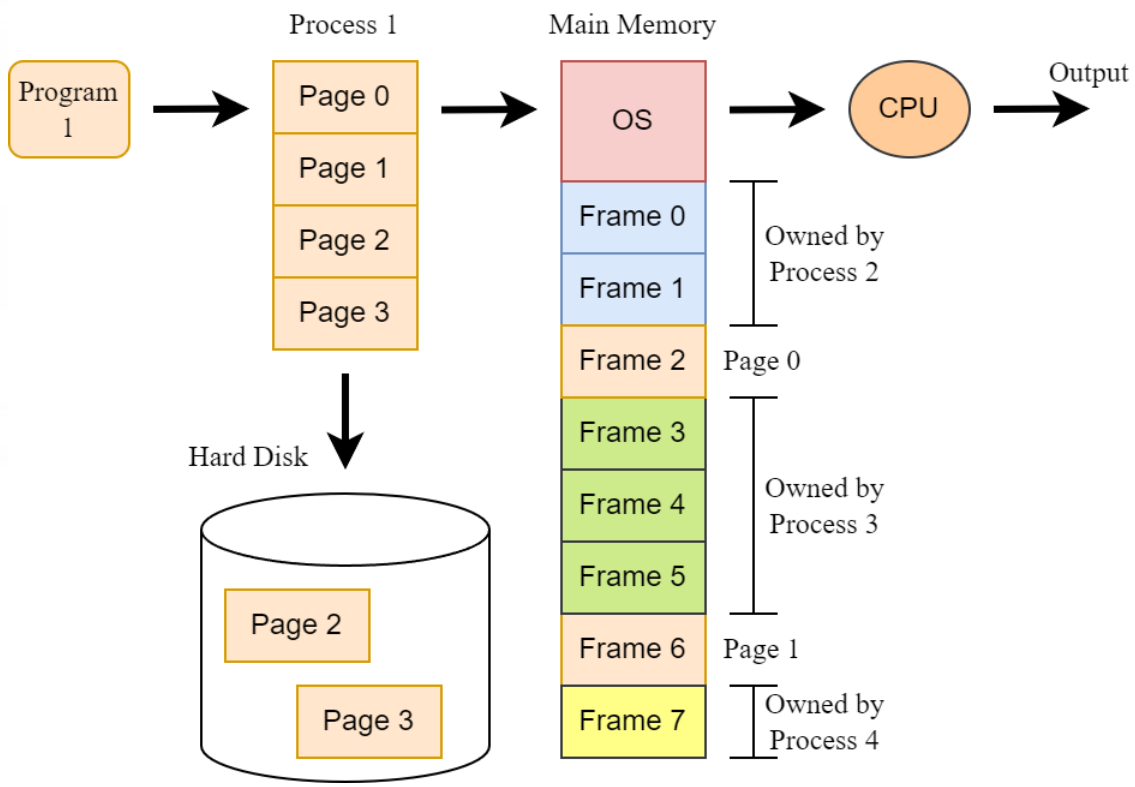
- If the CPU would like to execute Page 2 (stored in the hard disk) of the process P1, the page replacement is needed.



Example: Global replacement vs. local replacement

- We will show the difference between the global and local replacement.
  1) Global page replacement: To store Page 2 moved from the hard disk, the OS can free any frame (no matter which process currently owns it).
  2) Local page replacement: To store Page 2 moved from the hard disk, the OS can free only Frame 2 and Frame 6 that are currently owned by the process P1.





3. Page-Replacement Algorithms and Trashing

Scenario

While a user process is executing, a page fault occurs. The operating system must determine where the desired page is residing on the disk and move it to the memory.

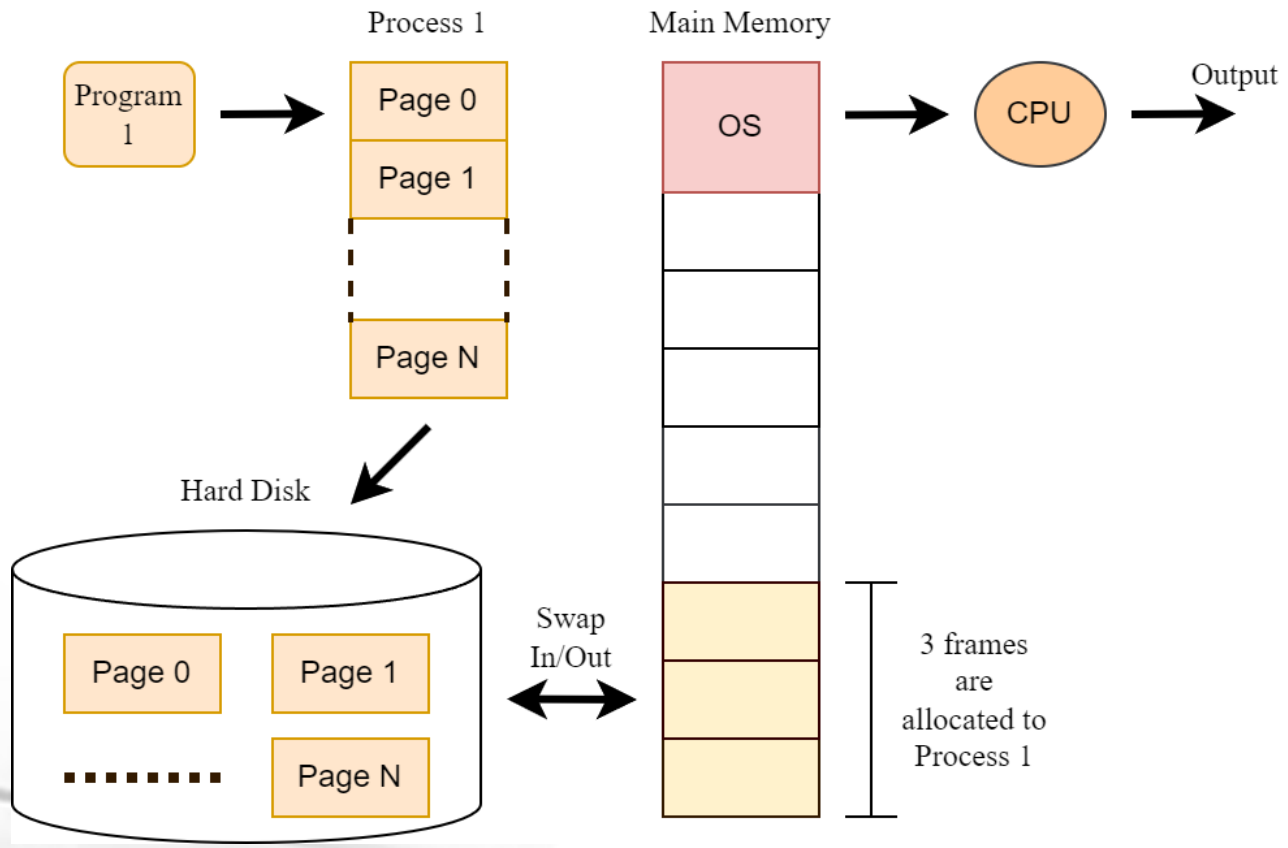
- A problem: If there is no free frame available, what should the operating system do?
- A solution: Page-replacement algorithms.

Page-Replacement Algorithms

- Objective:
  - To provide ways on which page in the memory should be swapped out to the hard disk when we have a new page coming into the memory.
- Three page-replacement algorithms are studied:
  - First-in first-out (FIFO) algorithm
  - Optimal algorithm
  - Least recently used (LRU) algorithm

Assumptions

1) We are considering page-replacement of a process.
   a) At the beginning, all pages are stored in the hard disk.
   b) Three frames in the main memory are allocated to this process. The pages can be stored only in these three frames (local page replacement).

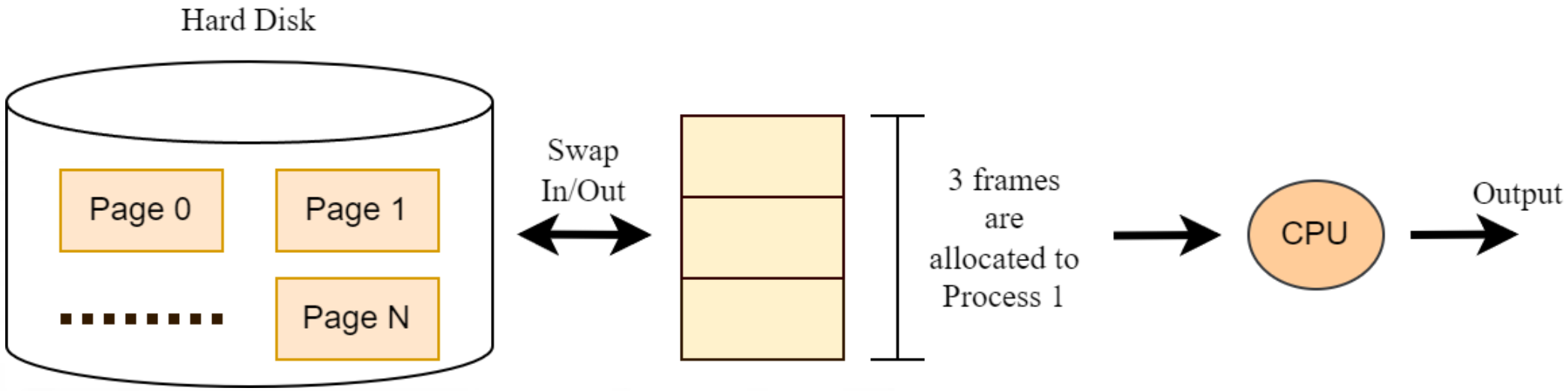


Assumption

2) Assume that the CPU execute the following sequence of page numbers (in order) which is called a reference string.

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

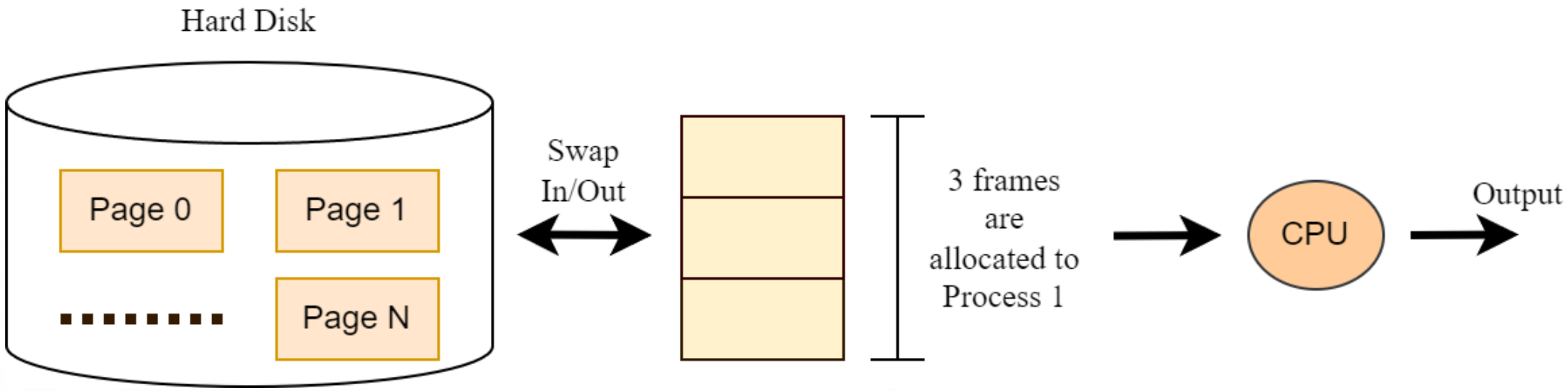
We use this reference string to evaluate and compare the performance of three methods.



Assumption

3) The performance of an algorithm is measured by the page-fault rate.

- Page fault rate is equal to the number of page faults.
  - 1 page fault = The CPU would like to execute a page in the hard disk.
- The lower number of page faults, the better performance. (A page fault induces an overhead time.)



1. FIFO (First-In, First-Out) Page-Replacement Algorithm

Principle:

"When a page must be replaced (swapped out to the disk), we choose (free) the oldest page staying in the memory out to the hard disk."

Example: FIFO Page-Replacement Algorithm

Consider that three frames are given in order to store pages from a process called P1. To run P1, the CPU will execute the following reference string (i.e., sequence of page numbers):

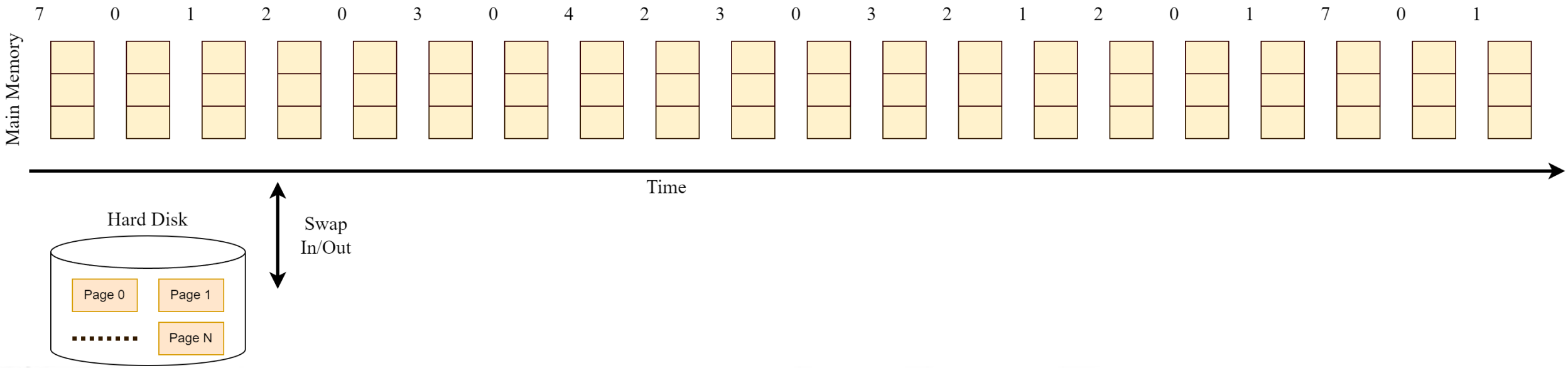
7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

Find the page fault rate by using the FIFO page-replacement algorithm.

- Answer: Including the first three times when new coming-in pages are loaded into three empty frames, there are 15 page faults.

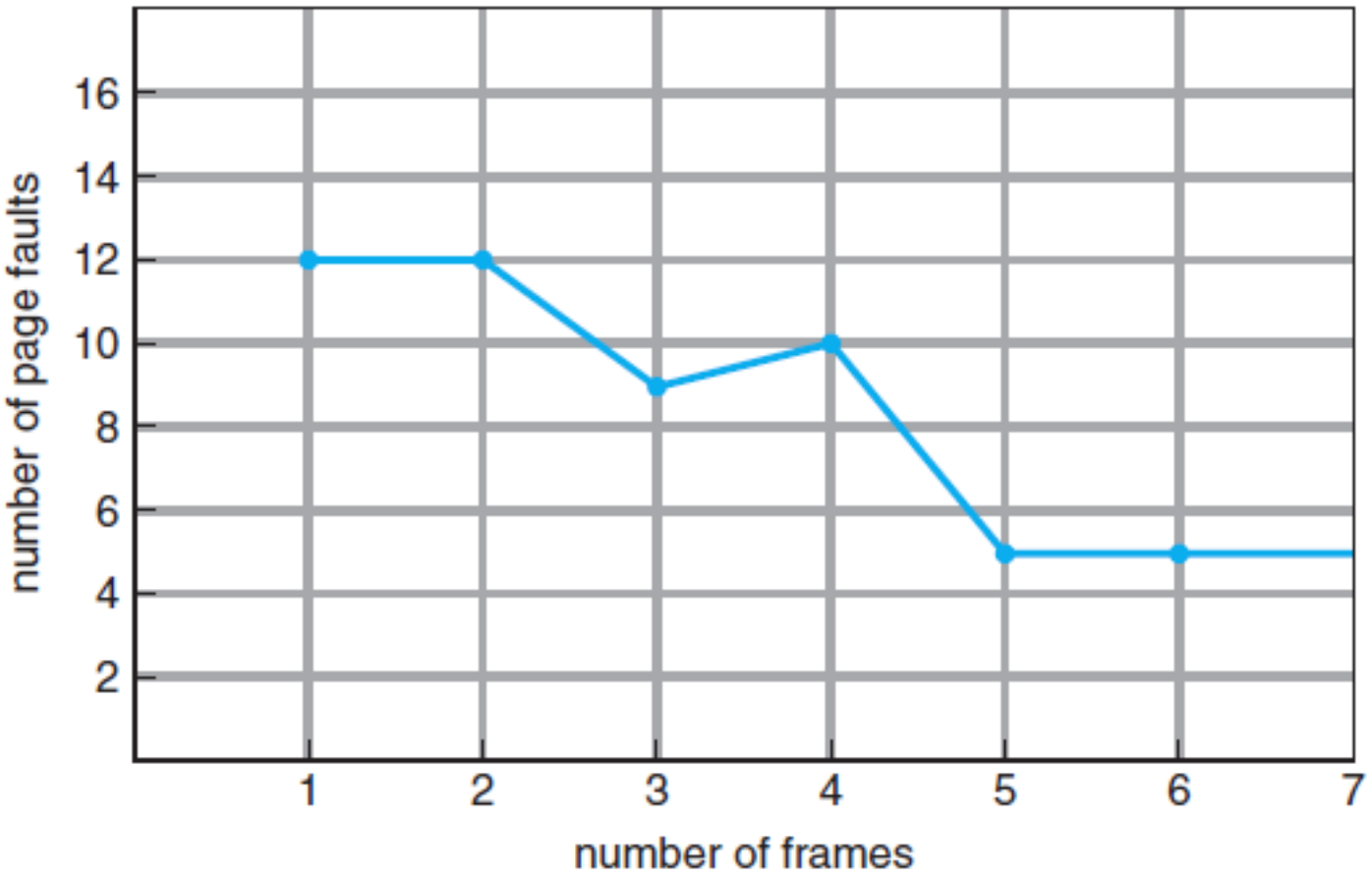
Example: FIFO Page-Replacement Algorithm

Solution:



FIFO's Problem: Belady's Anomaly

- Intuitive Idea:
  - If we increase the number of available frames, the page fault should be lower.
  - This is not true.
- Belady's anomaly
  - For FIFO page-replacement algorithms, the page-fault rate may increase as the number of allocated frames increases.



2. Optimal Page-Replacement Algorithm

- Principle:

  "When a page must be replaced (swapped out to the disk), we choose (free) the page that will not be used for the longest period of time." (Refer to the future usage.)

- Assumption:
  - We assume that the operating system knows the reference string.
- Key advantages:
  - Use of this page-replacement algorithm guarantees the lowest possible page fault rate for a fixed number of frames.
  - The optimal algorithm does not suffer from Belady's anomaly.
- Key disadvantage:
  - It is unlikely/impractical that we will know the entire reference string in advance.

Example: Optimal Page-Replacement Algorithm

Consider that three frames are given in order to store pages from a process called P1. To run P1, the CPU will execute the following reference string (i.e., sequence of page numbers):

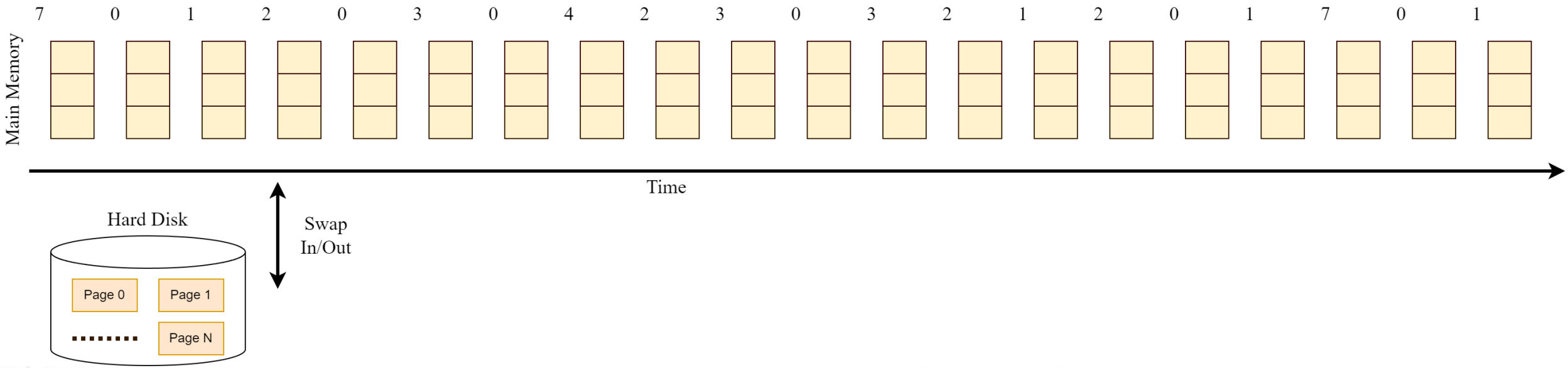
7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

Find the page fault rate by using the optimal page-replacement algorithm.

- Answer: Including the first three times when new coming-in pages are loaded into three empty frames, there are 9 page faults.

Example: Optimal Page-Replacement Algorithm

Solution:



3. Least-Recently-Used (LRU) Page-Replacement Alg.

- Principle:

  "When a page must be replaced (swapped out to the disk), we choose (free) the page that has not been used for the longest period of time." (Refer to the past usage.)

- Key advantages:
  - The LRU algorithm is practical and popular.
  - The LRU algorithm does not suffer from Belady's anomaly.
- Key disadvantage:
  - Implementing the LRU algorithm needs additional hardware to memorize the time when each frame/page has been recently used.

Example: LRU Page-Replacement Algorithm

Consider that three frames are given in order to store pages from a process called P1. To run P1, the CPU will execute the following reference string (i.e., sequence of page numbers):

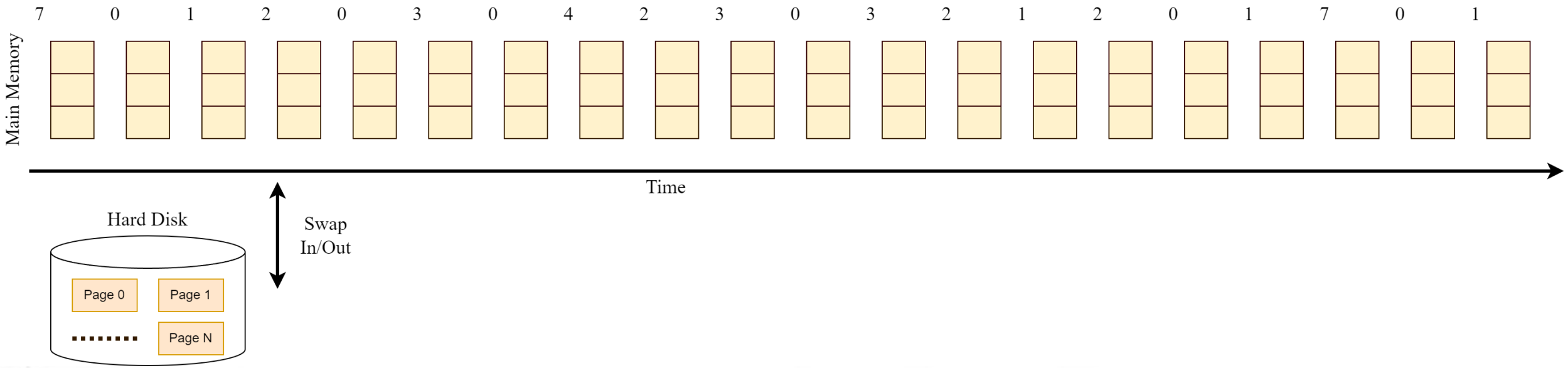
7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

Find the page fault rate by using the LRU page-replacement algorithm.

- Answer: Including the first three times when new coming-in pages are loaded into three empty frames, there are 12 page faults.

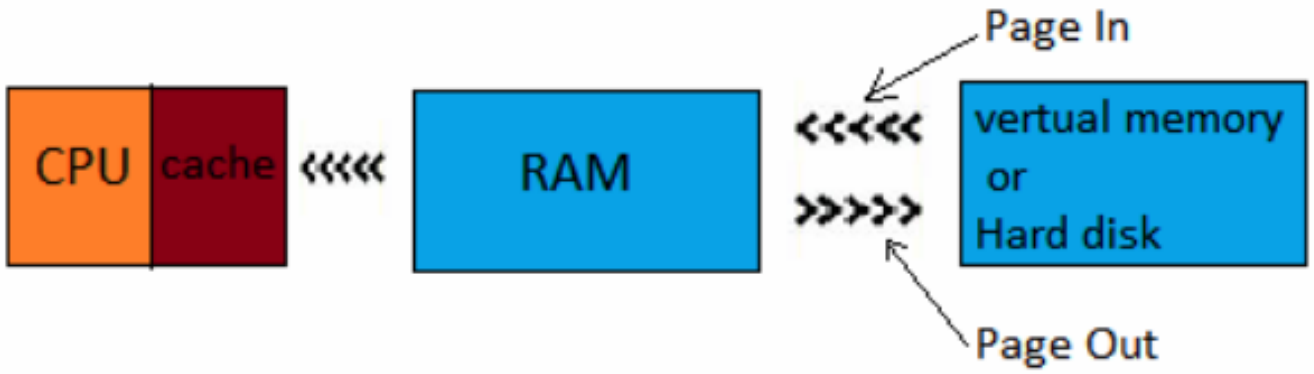
Example: LRU Page-Replacement Algorithm

Solution:



Thrashing from High Page-Fault Rate

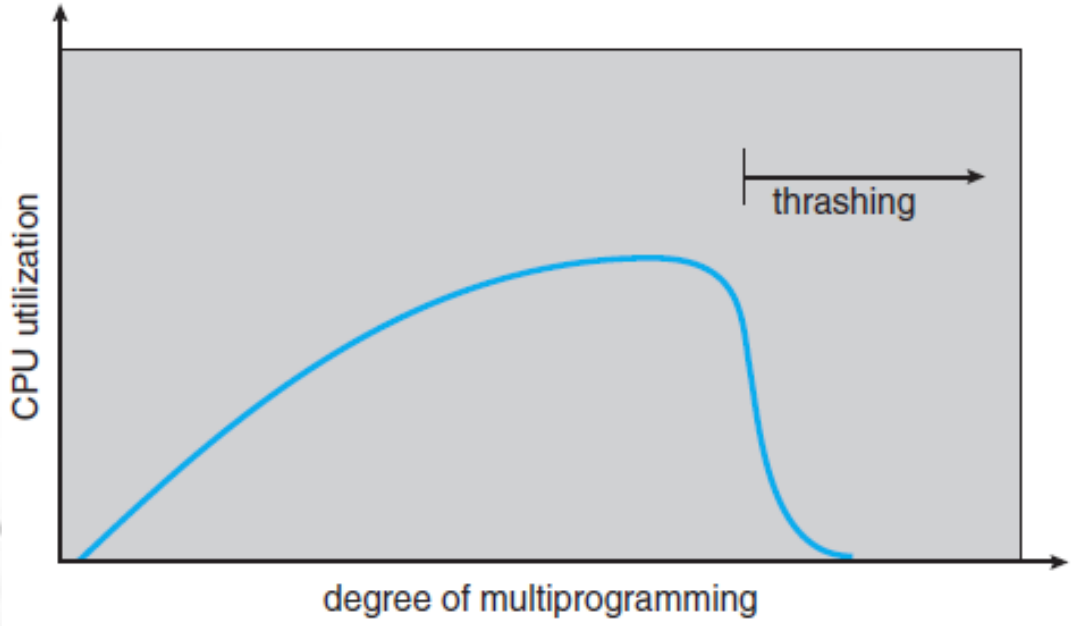
1) Thrashing is the situation that a system has to swap pages (between the memory and the disk) at such a higher number of page faults that most of the CPU time is spent in swapping (instead of running processes).
2) This situation mainly happens when most of the frames are being used and few of them can be swapped in/out. Then, these frames will be reused again and again.
3) As a result, a computer system that is thrashing can be perceived as either a very slow system or one that has come to a halt.



Thrashing: A Cause of Thrashing

Increasing the degree of multiprogramming (= increase the number of processes, to increase CPU utilization) beyond a particular point can cause trashing.

- Increasing the number of processes stored in both main memory and hard disk.
  - Since the number of frames in the main memory is limited, each process will have only few pages stored in the main memory and most of them stored in the hard disk.
  - As a result, increasing the page-fault rate.



4. Frame Allocation

Problem Statement

- We consider at the situation at the starting point of a system when all frames in the user space (in the main memory) are free.
- We know a number of processes loaded into the memory.
- We concern about how many free frames will be allocated to each process and how many should be reserved for future page-fault usage.

Allocation Algorithms

There are two simple frame-allocation algorithms.

1. Equal allocation: We split m frames among n processes equally. The leftover frames will be free frames for page faults.

Example: Equal Allocation Algorithm

If there are 93 free frames and 5 processes.

1) How many frames are allocated to each process?
2) How many frames are free for page-faults usage?

Allocation Algorithms

2. Proportional allocation: We allocate available frames to each process according to its size.
   - Let the size of the virtual memory for process p_i be s_i. Define $S = \sum_i^n s_i$, where n is the total number of processes.
   - Let m be the total number of available (free) frames.
   - We will allocate a_i frames to the process p_i where $a_i = \left\lfloor \left(\frac{s_i}{S}\right) \times m \right\rfloor$, where $\lfloor \quad \rfloor$ is the floor function.

Example: Proportional Allocation Algorithm

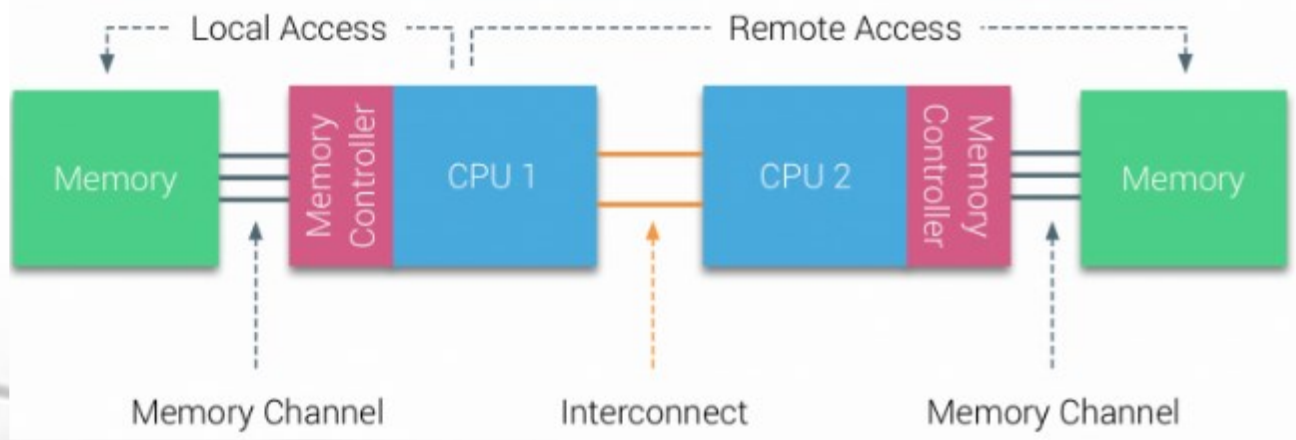
We would like to split 62 frames between 2 processes (called P1 and P2) where P1 has the size of 10 pages and P2 has the size of 127 pages.

1) How many frames are allocated to each process?
2) How many frames are free for page-faults usage?

5. Other Related Topics in Memory Management

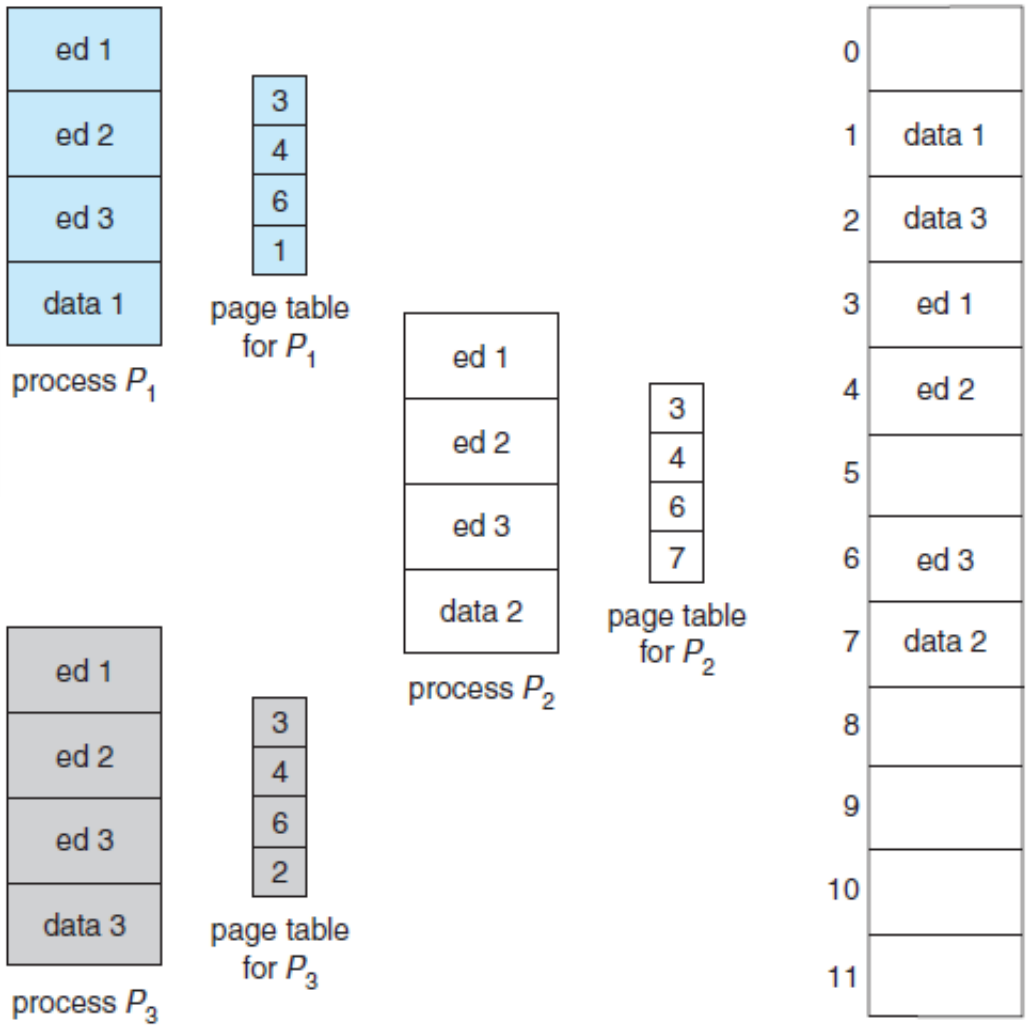
Non-Uniform Memory Access

- Non-uniform memory access (NUMA) is a shared memory architecture used in today's multiprocessing (multiple CPUs) systems.
  - Each CPU is assigned its own local memory and can access memory from other CPUs in the system.
    - Local memory access provides a low latency – high bandwidth performance.
    - While accessing memory owned by the other CPU has higher latency and lower bandwidth performance.
- Therefore, a process should be allocated the frames in the memory that belongs to the process's running CPU.



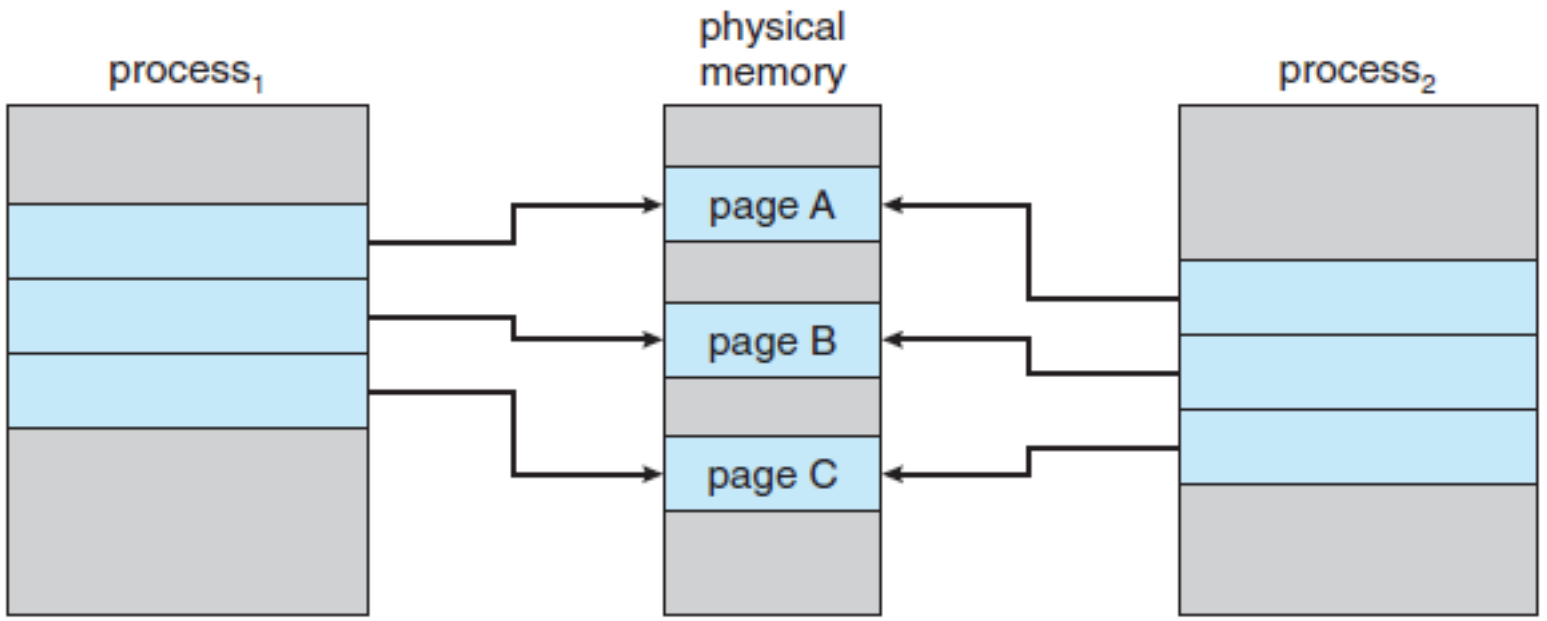
Shared Pages and Copy-on-Write Pages

1) Many processes might have similar operations and segment of codes.
   - As a result, some pages of these processes might be identical.
   - When mapping to frames (physical address), these processes might share the same frames.
2) These shared pages are marked as copy-on-write pages.
   - To edit a shared page, the process will make a copy of this page and store it into a new frame. Thereafter, the process can edit on the content of this page stored in the new frame.

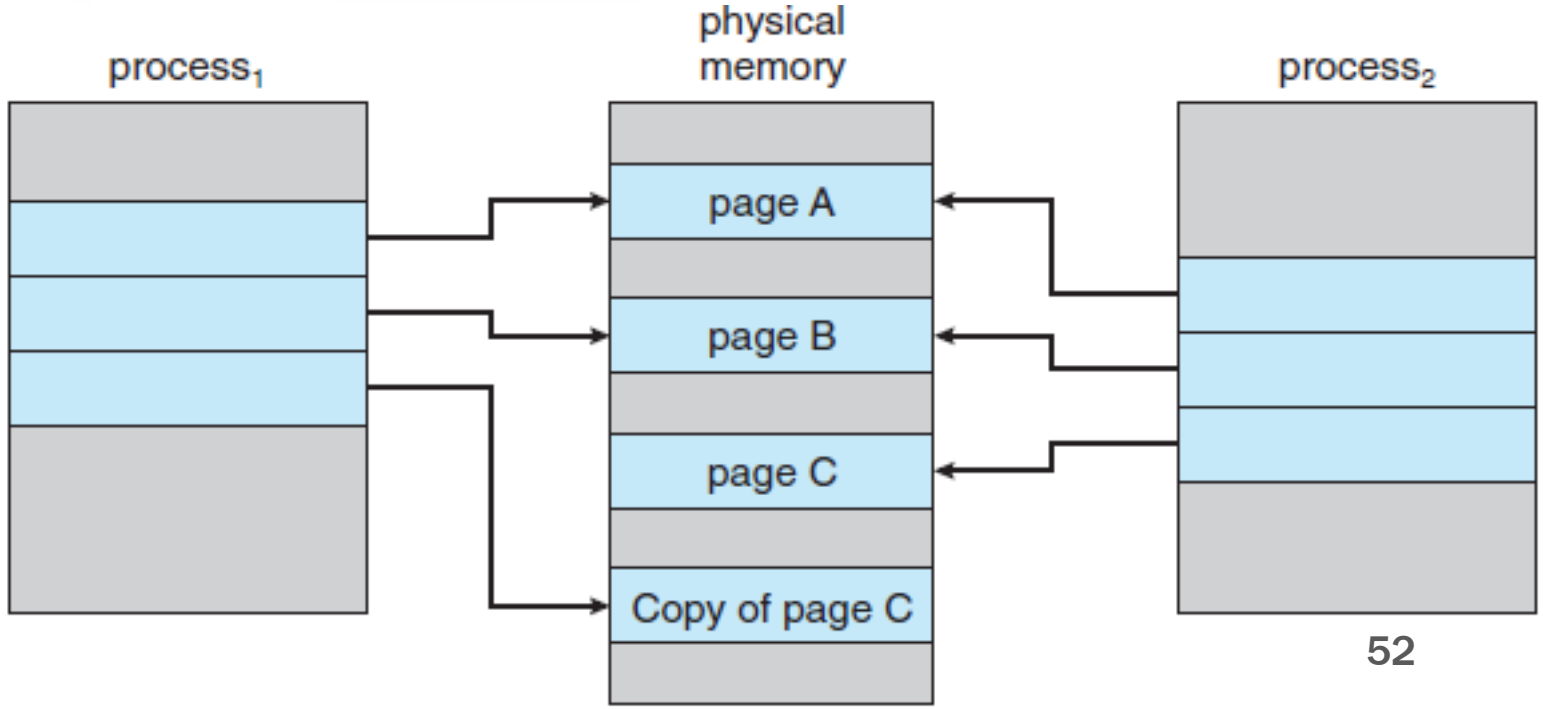


Example: Copy-on-Write

Consider that Process1 and Process2 are having the following shared pages (Pages A, B, and C).



If Process1 would like to edit its Page C, Process1 needs to make a copy of Page C and store it in a new frame before editing the content inside the Page C.



Reading Assignment

- Section 9.1, 9.2-9.2.1, 9.3, 9.4-9.4.4, 9.5, 9.6-9.6.1 in the textbook.

Glossary (by Google NotebookLM)

- Virtual Memory: A memory-management technique that allows the execution of processes that are not entirely in main memory.
- Demand Paging: A technique in virtual memory where pages are loaded into main memory only when they are actively needed by the CPU.
- Pager: A component responsible for transferring pages between main memory and the hard disk (swap space) in a demand-paging system.
- Swap Space: A designated area on the hard disk used to store pages that are part of a process but are not currently in main memory.
- Page Fault: An event that occurs when the CPU attempts to access a page that is not currently loaded into main memory.
- Page Replacement: The process of selecting a page in main memory to be removed (swapped out to disk) to make space for a new page that needs to be loaded.
- Valid-Invalid Bit: A bit in a page table entry that indicates whether the corresponding page is currently present in main memory (valid) or not (invalid).

Glossary (by Google NotebookLM)

- Reference String: A sequence of page numbers accessed by a process, used to evaluate the performance of page-replacement algorithms.
- Page-Fault Rate: The number of page faults, used as a metric for the performance of memory management.
- FIFO (First-In First-Out): A page-replacement algorithm that replaces the oldest page in memory.
- Optimal Algorithm: A page-replacement algorithm that replaces the page that will not be used for the longest time in the future.
- LRU (Least Recently Used): A page-replacement algorithm that replaces the page that has not been used for the longest time in the past.
- Belady's Anomaly: A phenomenon where increasing the number of available frames results in an increase in the page-fault rate for some page-replacement algorithms (like FIFO).
- Thrashing: A state in which a system spends an excessive amount of time swapping pages between main memory and disk, leading to poor performance.
- Global Replacement: A page-replacement strategy where the operating system can choose any frame in the system as a victim for replacement.

Glossary (by Google NotebookLM)

- Local Replacement: A page-replacement strategy where a process can only choose a victim frame from its own set of allocated frames.
- Equal Allocation: A frame allocation algorithm that distributes available frames equally among all processes.
- Proportional Allocation: A frame allocation algorithm that allocates available frames to processes based on their relative sizes.
- NUMA (Non-Uniform Memory Access): A shared memory architecture in multiprocessing systems where accessing memory local to a CPU is faster than accessing memory belonging to other CPUs.
- Copy-on-Write: A technique used for shared pages where a copy of the page is created only when one of the sharing processes attempts to modify it.